For Integration Developers

Prebuilt Integration Creation Guide

This document outlines the process that needs to be followed when developing a prebuilt integration.Tool Stack:

Tool Stack

- VSCode BI Extension(Devant Cloud Editor)
- Low-Code Environment
- AI Copilot

Environment Guidelines

- Cloud-First Policy: All integrations must begin in the Cloud Editor with AI assistance.
- The "Stop" Rule: Switching to Local/Pro-Code is only permitted after an Issue Ticket is opened and approved.

Step 1: Develop the integration

- Once the scenario is finalized, decide on the configurables that we are going to expose. For example, for an integration that exports all Salesforce Opportunities to a google sheet, possible configurables would be,
    - Timeframe
    - Whether to use an existing spreadsheet or create a new one
- Use the WSO2 Integrator: BI to develop the integration on the WSO2 Integration Platform Cloud Editor. Take advantage of the BI Copilot with the low-code mode.
- Always use Ballerina Connectors available in central.ballerina.io.
- If you encounter problems when trying to develop using either the copilot or low-code mode, please create an issue in wso2/product-integrator. You may switch to pro-code mode in those cases, after consulting the WSO2 Integration Platform (SaaS) team.
- Try to use the low-code mode as much as possible to detect usability issues. It will be beneficial to create a separate doc outlining the issues encountered.
- When using the pro-code mode, make sure you follow the Ballerina Coding Conventions.
- Once development is complete, thoroughly test the integration and ensure that edge cases are handled gracefully.
- In the `Ballerina.toml`, ensure the following are set.

```
None
[package]
org = "wso2"
name = "new_shopify_customer_to_stripe"
version = "0.1.0"
title = "new-shopify-customer-to-stripe"
distribution= "2201.13.1"
```

    - Ensure the `org` is set to `wso2`
    - `name` must clearly represent the purpose of the integration, without being lengthy. This name must be identical to the directory name, except the delimiter should be underscore (_) instead of the dash (-). This is **CRITICAL**, or else the build process fails.
    - Set the `version` to `0.1.0`
    - For the `title`, it is okay to use dashed delimited version of the `name`
    - Ensure the `distribution` is set to the latest available version

Step 2: Build the integration

Use the below command to build an Automation type integration. This will generate the `config-schema.json` file which will be needed in Step 3.

```
None
bal build --generate-config-schema --cloud docker
```

Use the below command to build an Integration as API type Integration. This will generate the `config-schema.json` file and the `openapi.yaml` file which contains the OpenAPI specification, which will be needed in Step 3.

```
None
bal build --generate-config-schema --export-openapi --cloud
docker
```

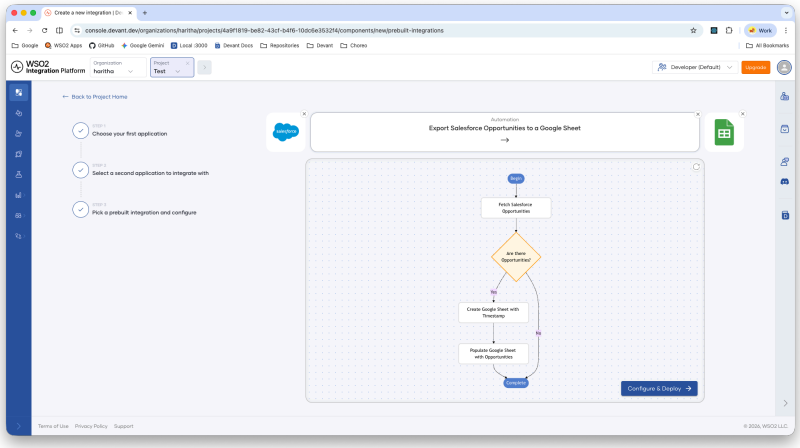
Please note that the `--cloud docker` flag is used to ensure that the build completes smoothly without dependency conflicts when building the image in the GitHub Workflow. Artifacts generated from it are not needed.

Step 3: Document the integration

- To deliver the best experience of a prebuilt integration, proper documentation is required. Please make sure the following documentation is done before making the pull request.
- Create the `README.md` file at the root of the directory. The structure of the file should be similar to [this example]().
- Create a `.choreo` directory at the root of the integration, and add the following files.

    - .choreo/diagram.md

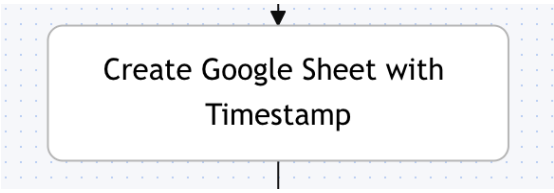
- This markdown file contains the flow chart defined in mermaid syntax that is rendered when selecting the prebuilt integration in the WSO2 Integration Platform.



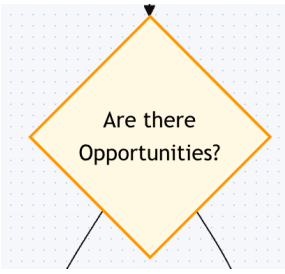
- This needs to use the below node styles appropriately.
  - startNode: Denotes a start node

  - endNode: Denotes an end node

  - processNode: Denotes a process node



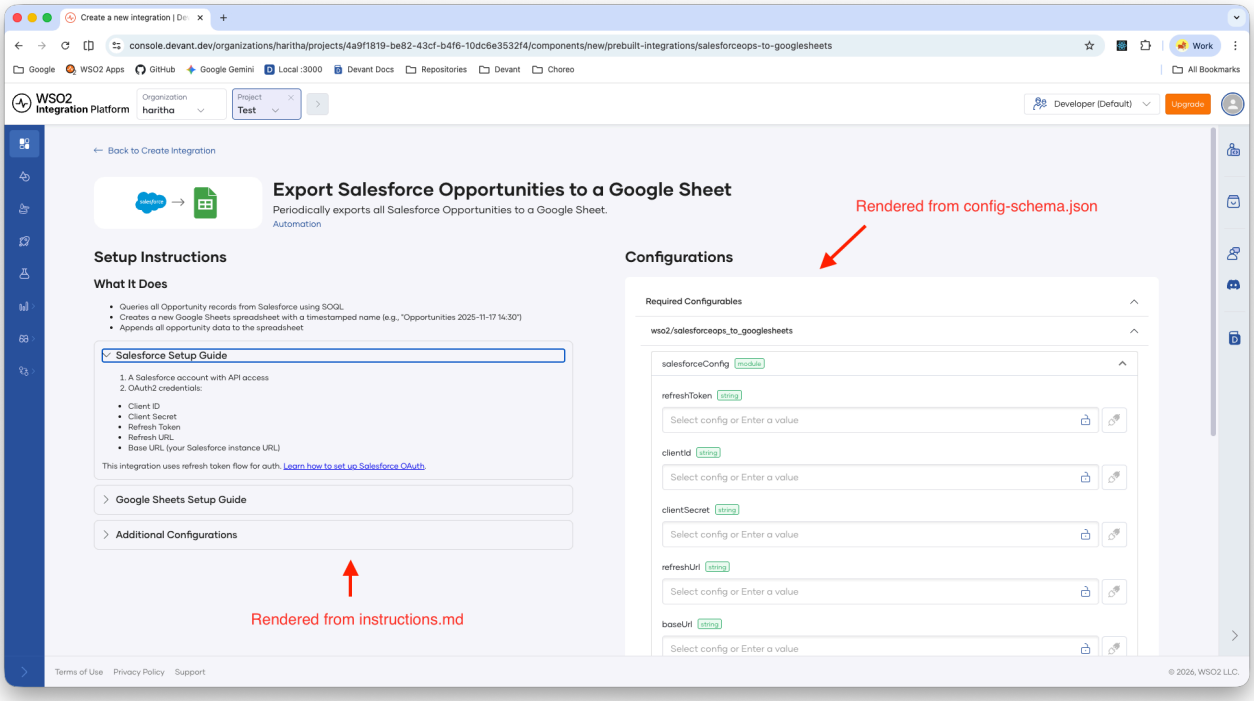
  - decisionNode: Denotes a decision node



  - Refer to this example.
- .choreo/instructions.md

    - This markdown file is rendered during prebuilt integration setup in the WSO2 Integration Platform.
    - Refer to this example. Ensure the structure is similar.
- .choreo/config-schema.json
    - This JSON file is used to render the configuration form during prebuilt integration setup in the WSO2 Integration Platform.
    - This has to be generated with the `bal build` command, which was outlined in Step 2.
    - In the generated JSON object, please add the following key-value pair to the end of the JSON object.

```JSON
"requiredLevel": 3
```



- .choreo/component.yaml
    - This is required only for Integration as APIs and AI agents.
    - Refer to this example.
- .choreo/openapi.yaml
    - This is the generated openapi specification with the `bal build` command, as outlined in Step 2.

Step 4: Delete unnecessary artifacts

- Ensure that the `target` directory is not pushed, as well as the `Dependencies.toml` file. Also ensure that the `target` directory is properly ignored in the `.gitignore`.

Step 5: Make the Pull Request

The pull request must be made to the [wso2/integration-samples]() repository. Prebuilt integrations live inside the `ballerina-integrator` directory.

PR Reviewers

PR Review Checklist

In addition to the general review process, please ensure the below. Integration developers can also use this as a checklist before submitting the PR.

- [ ] .github/workflows/projects.json is updated with the new prebuilt integration path
- [ ] Path in the .github/workflows/projects.json matches the project path exactly
- [ ] Directory path is identical to the integration name in `Ballerina.toml`
- [ ] README.MD is correctly structured
- [ ] .choreo/config-schema.json exists and is up to date
- [ ] .choreo/choreo-config-schema.json exists and is up to date
- [ ] .choreo/instructions.md is correctly structured
- [ ] .choreo/diagram.md is correctly written and parsable by Mermaid
- [ ] .choreo/component.yaml exists for service type integrations
- [ ] .choreo/openapi.yaml exists for service type integrations